

Exercise 1: Unit propagation (BCP) of DPLL

In this exercise, you will use unit propagation (Boolean Constraint Propagation) to compute the level-0 assignment in DPLL/CDCL. Level 0 contains only consequences of unit clauses (and units created during propagation); do not make any decisions and do not use pure-literal elimination.

For each CNF below:

- Start with an empty trail at decision level 0.
- Repeatedly propagate unit clauses until you reach a fixpoint or a conflict.
- List the sequence of implied literals and the final level-0 assignment.
- If a conflict is detected at level 0, state that the formula is unsatisfiable at level 0 (top-level conflict) and identify the clause(s) involved.

Instance 1 Variables: a, b, c, d, e, f, g .

$$\begin{aligned}\Phi_1 = & (a) \\ & \wedge (\neg a \vee b) \\ & \wedge (\neg a \vee \neg b \vee d \vee f) \\ & \wedge (\neg b \vee \neg c)\end{aligned}$$

Tasks:

- Show the propagation sequence (e.g., $a \Rightarrow b \Rightarrow \dots$).
- Give the final set of implied literals at level 0.
- Indicate whether any variables remain unassigned after propagation.

Instance 2 Variables: a, b, c .

$$\begin{aligned}\Phi_2 = & (a) \\ & \wedge (\neg a \vee b) \\ & \wedge (\neg b \vee c) \\ & \wedge (\neg c)\end{aligned}$$

Tasks:

- Run unit propagation from the empty trail. If a conflict arises, report it and list the last few implications that led to the conflict.
- State clearly whether there is a top-level conflict (i.e., the formula is already inconsistent at level 0 without any decisions).

Instance 3 Variables: a, b, c, d, e .

$$\begin{aligned}\Phi_3 = & (a) \\ & \wedge (\neg a \vee b) \\ & \wedge (\neg a \vee c) \\ & \wedge (\neg b \vee \neg c \vee d) \\ & \wedge (\neg d \vee e)\end{aligned}$$

Tasks:

- Show the propagation sequence (e.g., $a \Rightarrow b \Rightarrow \dots$).
- Give the final set of implied literals at level 0.
- Indicate whether any variables remain unassigned after propagation.

Instance 4 Variables: a, b, c, d, e .

$$\begin{aligned}\Phi_4 = & (a) \\ & \wedge (\neg a \vee b) \\ & \wedge (\neg b \vee c \vee d) \\ & \wedge (\neg c \vee e) \\ & \wedge (\neg d \vee e) \\ & \wedge (\neg e)\end{aligned}$$

Tasks:

- Run unit propagation from the empty trail. If a conflict arises, report it and list the last few implications that led to the conflict.
- State clearly whether there is a top-level conflict (i.e., the formula is already inconsistent at level 0 without any decisions).

Exercise 2: Implementing DPLL

Implement the DPLL algorithm in `dp11.py`.

Exercise 3: Implementing Simplex

Implement the Simplex algorithm in `simplex.py`.

Exercise 4: SMT encoding of a neural network

Write an SMT encoding for a simple neural network. You may use any SMT solver you like (for example, Z3).

- Choose a small feed-forward neural network (you may design your own with a few layers and ReLUs).
- Encode its linear and ReLU constraints in SMT (e.g., using Z3Py).
- Formulate and check at least one simple property, such as robustness within a small ϵ -ball or a bound on an output neuron.

If you do not know where to start with Z3, you can look at the Z3Py examples online (for example, the Z3Py tutorial by Eric Pony).